



UNIVERSITY OF BUCHAREST

**FACULTY OF MATHEMATICS
AND COMPUTER SCIENCE**



COMPUTER SCIENCE PROGRAM

Bachelor's Thesis



COMPILER OPTIMIZATION THROUGH MACHINE LEARNING TECHNIQUES

Graduate

Ingrid-Adriana Corobană

Scientific Advisor

Prof. Dr. Cristian Rusu

Bucharest, 2026

Abstract

Compiler inlining is treated as an offline policy-learning problem. In the proposed pipeline, C++ sources from a local DCMTK snapshot are compiled with Clang to LLVM IR using inlining disabled, and each callsite is converted into a feature vector that captures local and graph-level context. A small set of inspectable teacher heuristics is then evaluated offline on every program, the best-performing teacher is selected per module, and its callsite decisions are turned into a supervised dataset. Student models are trained with behavior cloning to imitate the selected teacher trajectory, then their predicted decisions are applied back to the LLVM IR through a callsite-level rewrite. The thesis evaluates the resulting student policies using rewritten IR instruction count and compares the pipeline against simple native code-size baselines, showing how compiler optimization can be framed as a practical machine-learning workflow rather than a hand-written heuristic alone.



Contents

1	Introduction	1
1.1	Research Questions	1
1.2	Contributions	2
2	Compiler Background	3
2.1	Compilation Pipeline	3
2.2	LLVM and LLVM IR	4
2.3	Static and Dynamic Measurements	4
3	Function Inlining	5
3.1	Cases Where Inlining Helps	5
3.2	Cases Where Inlining Hurts	6
4	Problem Formulation	8
5	Related Work	10
5.1	MLGO	10
5.2	Offline Imitation Learning and BC-Max	11
6	Methodology	12
6.1	Dataset Summary	12
6.2	Teacher Policies	13
7	Implementation	14
7.1	Student Model Training	14
8	Evaluation	15
9	Limitations	20
9.1	Small Dataset	20
9.2	No Held-Out Generalization Result	20
9.3	Textual IR Rewriting	20
9.4	IR Instruction Count Versus Native Binary Size	20

9.5	Teacher Quality	21
9.6	No Full MLGO Reproduction	21
10	Conclusions	22
	Bibliography	23

Chapter 1

Introduction

Classical compilers usually rely on manually designed heuristics for inlining. These heuristics try to balance several competing goals: reduce call overhead, expose later optimizations, avoid unnecessary code growth, and protect the instruction cache. The goal of this thesis is to study whether a learned policy can imitate useful teacher heuristics and make inline/no-inline decisions from compiler features.



Function inlining is a suitable case study because it is both local and global. Locally, an inlining decision concerns one callsite: a caller invokes a callee, and the compiler must decide whether to replace the call by the body of the callee. Globally, this decision changes the program representation. The caller may grow, the callee may become removable, the call graph may change, and later optimization passes may simplify or delete newly exposed code. Therefore, the effect of one local decision is not always visible until later stages of compilation.

This thesis is inspired by *Offline Imitation Learning from Multiple Baselines with Applications to Compiler Optimization*, especially its idea of selecting the best teacher policy per program and training a student policy from the selected trajectories [1]. The implemented prototype follows the same high-level pattern at a smaller scale: teacher heuristics produce candidate inlining trajectories, the best teacher is selected per module according to a size objective, and student classifiers are trained through behavior cloning.

The evaluation rewrites LLVM IR with LLVM’s `opt` tool and measures rewritten IR instruction counts. IR instruction count is easy to inspect, architecture-independent, and closely connected to the effect that inlining has on the compiler representation. Native object `.text` size is also measured as a secondary reference point, but the learned student policy is evaluated primarily through rewritten LLVM IR instruction count.

1.1 Research Questions

This thesis is guided by the following research questions:

- **RQ1.** Can compiler inlining be represented as an offline imitation-learning problem from

teacher heuristics?

- **RQ2.** Can a BC-Max-style selection rule construct a labeled callsite dataset by selecting the best teacher trajectory per module?
- **RQ3.** Can simple student models imitate the selected teacher decisions from callsite-level features?
- **RQ4.** Does applying the learned student decisions reduce rewritten LLVM IR instruction count compared with the never-inline baseline on the current dataset?

1.2 Contributions

The main contributions of this thesis are:

- a small end-to-end pipeline that compiles C++ files to LLVM IR, extracts callsite features, evaluates teacher heuristics, trains student models, and rewrites LLVM IR according to predicted decisions;
- a formalization of inlining as a sequential policy-learning problem;
- an implementation of a BC-Max-inspired offline imitation-learning workflow for compiler inlining;
- an experimental evaluation on a small DCMTK-based source snapshot using rewritten LLVM IR instruction count and native `.text` size as code-size measurements;
- a discussion of the limitations of textual IR rewriting, small datasets, and the gap between a prototype and a full LLVM inliner integration.

Chapter 2

Compiler Background

2.1 Compilation Pipeline

Compilation transforms a high-level program into representations that are easier for a machine to execute. A simplified LLVM-based pipeline is:

C++ source

-> Clang frontend

-> LLVM IR (.ll)

-> optimized LLVM IR



-> machine-specific backend

-> assembly text (.s or .asm, optional visible output)

-> assembler

-> object files (.o or .obj)

-> linker

-> executable file

Clang parses C++ source code, checks the language rules, and emits LLVM IR. The IR stage is still abstract enough to be mostly target-independent, but it is lower-level than C++ and contains operations that are easier for the compiler to analyze and optimize. After IR optimization, the backend becomes machine-specific. It chooses target instructions, performs register allocation, schedules instructions, and prepares code for a concrete architecture.

Most optimization passes are applied to the LLVM IR rather than to the source or the final object code. That is why this thesis focuses on IR instruction count as the main measurement, while object-file sizes are used only as a secondary reference point.

Assembly is a human-readable textual form of machine instructions. Compilers may emit it as an intermediate file, but many build pipelines go directly from the backend to object-file emission without leaving a visible assembly file on disk. The assembler converts assembly-level instructions into object files containing machine-code bytes, relocation entries, symbols, and linking metadata. Programs often have many source files, so they also often produce many

object files. The linker combines these object files and library references into a final executable.

Linking mostly happens at build time. Dynamic linking and loading can still happen partly at runtime when shared libraries are used. Link-Time Optimization (LTO) keeps IR available until link time, allowing cross-file optimizations such as inlining across translation units.

2.2 LLVM and LLVM IR

LLVM is a compiler infrastructure project. It includes libraries and tools for building compilers, including the LLVM optimizer and backends. Clang is the C, C++, and Objective-C frontend that targets LLVM. LLVM IR is not the whole LLVM project; it is the intermediate representation used by LLVM.

LLVM IR contains virtual registers, typed operations, branches, function calls, and memory operations. These virtual registers are not physical CPU registers. They are later mapped or lowered by the backend during instruction selection and register allocation. This makes LLVM IR a useful place to study inlining because the representation is lower-level than source code but still independent of a particular machine’s object-file format.

2.3 Static and Dynamic Measurements

Several size-related terms are useful when discussing inlining:

- A static call graph is the graph of possible caller-callee relationships visible from the program representation before choosing one concrete runtime path.
- A dynamic call graph is the graph of calls actually observed for a particular execution and input.
- Static code size is the size of the compiled program’s code, for example the size of code sections on disk or in memory.
- Dynamic instruction count is the number of instructions actually executed for a workload.

The phrase “dynamic code size” is less standard than dynamic instruction count. For this reason, the thesis uses static code size for generated code that exists in the compiled artifact, and dynamic instruction count for the amount of work actually executed under a particular workload.

Chapter 3



Function Inlining

Inlining is the compiler optimization that replaces a function call in the caller with the body of the callee. The original call instruction can then disappear, and the callee body becomes visible inside the caller.

Inlining can help for two main reasons. First, it removes call overhead: the work needed to transfer control to another function, prepare arguments, preserve state according to the calling convention, and return to the caller. Second, it exposes new optimization opportunities. Once the callee body is inside the caller, constants and known branches from the caller may simplify the callee.

3.1 Cases Where Inlining Helps

A small arithmetic helper is a typical good inlining candidate:

```
int ab(int a, int b) {  
    return a * b;  
}
```



```
int sum_ab(int a, int b) {  
    return a + b + ab(a, b);  
}
```

Inlining `ab` removes a function call and replaces it with a small expression. If this call is in a hot part of the program, the saved overhead can matter.

Inlining is also useful when it enables constant propagation:

```
string light_color(int c, int i) {  
    if (c == 0)  
        return light_red(i);  
    if (c == 1)  
        return light_green(i);  
}
```

```

        return light_blue(i);
    }

    string car_color(int i) {
        int f;
        return light_color(0, 2 + f);
    }

```

After inlining, the compiler can see that `c` is always zero at this callsite. The other branches can be removed, and the result can become equivalent to:

```

    string car_color(int i) {
        int f;
        return light_red(2 + f);
    }

```

3.2 Cases Where Inlining Hurts

Recursive functions are dangerous inlining candidates:

```

int sum(int x) {
    if (x <= 0)
        return 0;
    return x + sum(x - 1);
}

```

Inlining one recursive call only exposes another recursive call. Repeating this process can increase code size without reaching a useful fixed point. Classical compilers therefore use recursion checks and growth budgets.

Inlining can also be bad when a large function is rarely executed:

```

void game_engine_reinit() {
    // large recovery path
}

void entry(bool error) {
    if (error)
        game_engine_reinit();
}

```

The recovery function is on a cold path. Inlining it may increase static code size and pollute the instruction cache, while giving little benefit to the hot path.

Another bad case is repeated copying of a large callee:

```
int get_info(int id) {  
    for (...)  
        for (...)  
            return ...;  
}  
  
void info_candidates() {  
    cout << get_info(12) << endl;  
    cout << get_info(133) << endl;  
    cout << get_info(144) << endl;  
}
```

If `get_info` is large, copying it at every callsite can increase code size more than it saves in call overhead.



Chapter 4

Problem Formulation

Given a program or module P , let $S(P)$ be the set of callsites observed during the inlining pass. A callsite is a function-call location at which a caller function invokes a callee function. For each callsite, the compiler can extract a feature vector:

$$s = \phi(c),$$

where c is the callsite and ϕ is the feature-extraction function. In this thesis, such features include caller and callee information, local instruction counts, call counts, constant-argument information, and limited call-graph context.

The action space is binary:

$$A = \{0, 1\},$$

where

$$0 = \text{do not inline}, \quad 1 = \text{inline}.$$

A policy is therefore a function

$$\pi : \mathcal{S} \rightarrow A,$$

or, in the probabilistic case,

$$\pi(a \mid s),$$

where $\pi(a \mid s)$ denotes the probability of choosing action a in state s .

Inlining makes the problem sequential. If a policy π is applied to a program P , it generates a trajectory

$$\tau_\pi(P) = ((s_1, a_1), (s_2, a_2), \dots, (s_H, a_H)),$$

where H is the number of inlining decisions made for that program. After one callsite is inlined, the remaining callsites and their properties may change. The call graph can lose an edge, the caller can become larger, the callee may become dead if all its uses were inlined, and new simplifications may appear. Therefore, the decisions are not fully independent.

The objective is to minimize generated code size. In the implemented prototype, the main

size metric is the final rewritten LLVM IR instruction count:

$$\text{Size}_{IR}(P, \pi).$$

A lower value is better. Equivalently, one can define a reward:

$$R(P, \pi) = -\text{Size}_{IR}(P, \pi).$$

Let

$$H = \{h_0, h_1, \dots, h_k\}$$

be the set of teacher policies. Each teacher h_i maps callsite features to an inline/no-inline decision. For each training program P_j , every teacher is evaluated and the best teacher is selected by:

$$h^*(P_j) = \arg \min_{h_i \in H} \text{Size}_{IR}(P_j, h_i).$$

The selected teacher trajectory becomes supervised training data:

$$\mathcal{D} = \bigcup_j \{(s_{j,t}, a_{j,t}^*)\}_{t=1}^{H_j},$$

where $a_{j,t}^*$ is the action taken by the selected teacher $h^*(P_j)$ at state $s_{j,t}$.

A student model π_θ is trained by behavior cloning. In the ideal case, it minimizes disagreement with the selected teacher actions:

$$\min_{\theta} \sum_{(s,a) \in \mathcal{D}} \mathbf{1}[\pi_\theta(s) \neq a].$$

In practice, the implementation trains standard classifiers. For probabilistic models, the corresponding loss is the binary cross-entropy:

$$\mathcal{L}(\theta) = - \sum_{(s,a) \in \mathcal{D}} \log \pi_\theta(a \mid s).$$

After training, the student becomes a new heuristic. At each callsite, the student receives the current feature vector and predicts whether to inline:

$$\hat{a} = \arg \max_{a \in \{0,1\}} \pi_\theta(a \mid s).$$

The predicted decisions are then applied to the LLVM IR rewrite step and evaluated by the resulting IR instruction count.

Chapter 5

Related Work

5.1 MLGO

MLGO formulates compiler optimization as a problem where a learned policy replaces or guides a heuristic inside LLVM [3]. For inlining, the high-level reinforcement-learning formulation is:

- state: the current call graph and current callsite features;
- environment: the compiler;
- action: inline or no-inline;
- transition: deterministic compiler update after the action;
- reward: final improvement in the compiled program, commonly native code-size reduction for the inlining-for-size setting;
- policy: a model $\pi(a \mid s)$ that outputs an action probability.

In theory, the full call graph would be a rich state representation because it captures global changes. In practice, using the full state directly is expensive, so the compiler exposes a vector of features about the current callsite, caller, callee, and global graph. Examples include caller/callee basic block counts, callee users, callsite height, estimated cost, number of constant parameters, and graph node/edge counts.

The original MLGO inlining setup is trained as a policy-gradient problem, with PPO-style updates used to improve the policy from the final reward signal. The reward is delayed: the quality of a local inline decision is difficult to measure exactly when the decision is made, because later compiler passes and later inlining decisions may interact with it. This motivates reinforcement learning in MLGO, since the policy can be optimized from the final compilation outcome rather than from manually labeled callsite actions. However, this also makes full on-line reinforcement learning expensive, especially when each environment interaction requires compilation.

5.2 Offline Imitation Learning and BC-Max

The paper *Offline Imitation Learning from Multiple Baselines with Applications to Compiler Optimization* proposes a cheaper alternative to online reinforcement learning [1]. Instead of learning only from trial and error, it starts from a set of teacher policies:

$$H = \{h_0, h_1, \dots, h_n\}.$$

Each teacher policy receives callsite features and emits an inline/no-inline decision. For every training program j_i , all teachers are evaluated. The teacher with the best final objective value for that program is selected:

$$h_i^* = \arg \min_{h \in H} \text{size}(h, j_i).$$

The selected teacher trajectory is saved:

$$\tau_i = \{(x_0, h_i^*(x_0)), \dots, (x_m, h_i^*(x_m))\}.$$

The union of these trajectories becomes a supervised dataset. A student policy h_{n+1} is trained to predict the selected teacher’s decisions. This is the BC-Max idea: behavior cloning from the best baseline per program. The student can later be added back into the teacher set and the procedure can be repeated.

Teacher policies may contradict each other on similar local callsites. This is one reason feature design matters. When local features are not enough, graph context, callee-use counts, constant-argument information, and callsite height can help distinguish cases.

Chapter 6

Methodology

The pipeline uses this sequence:

C++ source

- > Clang -O0 -fno-inline
- > LLVM IR
- > callsite features
- > teacher policies
- > best teacher per module
- > labeled dataset
- > student classifiers
- > opt-based LLVM IR rewrite
- > rewritten IR instruction-count evaluation



The C++ sources come exclusively from DCMTK [2], copied from the local checkout used by the main pipeline. The snapshot contains 18 `config/tests/*.cc` files; in the current run, 17 are compiled successfully to LLVM IR and one fails because `lfs64.cc` references `fopen64`, which is unavailable on this local macOS toolchain.

The first eleven features follow the public shape of the MLGO inlining feature set: caller, callee, callsite, and global call-graph information. The model also adds interpretable local features: instruction counts, call counts, and a recursion indicator.

6.1 Dataset Summary

The current prototype uses a small DCMTK-based source snapshot. The dataset is not meant to be a representative compiler benchmark suite; it is used to test whether the offline imitation-learning pipeline can be implemented end to end.



Quantity	Value
Source project	DCMTK
Candidate C++ files	18
Successfully compiled files	17
Failed files	1
Extracted local callsites	40
Teacher policies	6
Student models	5
Primary metric	Rewritten LLVM IR instruction count
Secondary metric	Native object .text size

The small number of callsites is an important limitation of the current experiment. The results should therefore be interpreted as a proof of concept for the methodology rather than as evidence of a production-quality learned compiler heuristic.

6.2 Teacher Policies

The local teacher set is intentionally simple and inspectable:

- `never_inline`: never inline any local callsite;
- `small_callee`: inline small non-recursive callees;
- `single_caller`: inline callees that have only one observed local caller and remain under a size threshold;
- `constant_argument`: inline when constants may enable later simplification;
- `cost_budget`: inline under a simple estimated-cost budget;
- `loop_averse`: avoid branchy or call-heavy callees.

These teachers are simple baselines used in the offline imitation-learning workflow.

Chapter 7

Implementation

The implementation is located in the `scripts` directory. The code compiles the DCMTK source snapshot to LLVM IR, parses local callsites, extracts features, evaluates teacher policies, trains student models, and writes JSON and Markdown artifacts for inspection.

Run the full pipeline from the `thesis_attempt` directory with:

```
python3 scripts/pipeline.py all
```

The evaluated student models are logistic regression, decision tree, k-nearest neighbors, random forest, and a small MLP. Logistic regression is a linear classifier, decision tree learns rule splits, k-nearest neighbors predicts from nearby examples, random forest combines many trees, and the small MLP is a shallow neural network.

7.1 Student Model Training

The student models are trained on the behavior-cloning dataset produced by the BC-Max selection step. Each training example consists of a callsite feature vector and the binary action chosen by the selected teacher for that module. The label is therefore not an externally provided ground truth label; it is the action taken by the best teacher trajectory according to the rewritten IR size objective.

The trained models are then used as policies. During evaluation, the model is queried at each extracted callsite. If the predicted action is inline, the corresponding LLVM IR call edge is rewritten by cloning the callee, marking the clone as `alwaysinline`, and invoking LLVM `opt`. If the predicted action is no-inline, the callsite is left unchanged. The final rewritten IR is then counted to obtain the evaluation metric.

Chapter 8

Evaluation



The current run compiles 17 DCMTK C++ sources and extracts 40 local callsites. The numeric column named teacher-label match is computed from the generated behavior-cloning dataset:

$$\frac{\#\{\text{student prediction} = \text{selected teacher label}\}}{\#\{\text{callsites}\}}.$$

It is an imitation metric, not a comparison with LLVM’s built-in inlining heuristic and not a held-out generalization result. The evaluation focuses on rewritten LLVM IR instruction count and, as a secondary check, native .text size against Clang/LLVM -Oz.

The plots below summarize the current pipeline run. The first two show actual size-reduction outcomes, and the last one shows the student agreement metric used to produce the table below.

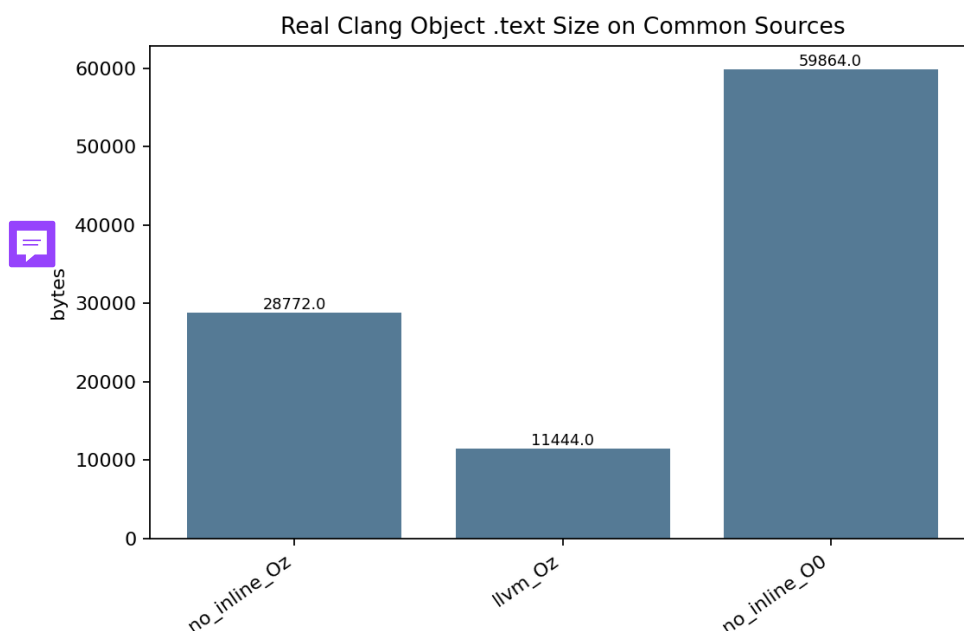


Figure 8.1: Real Clang object .text size on the common sources.

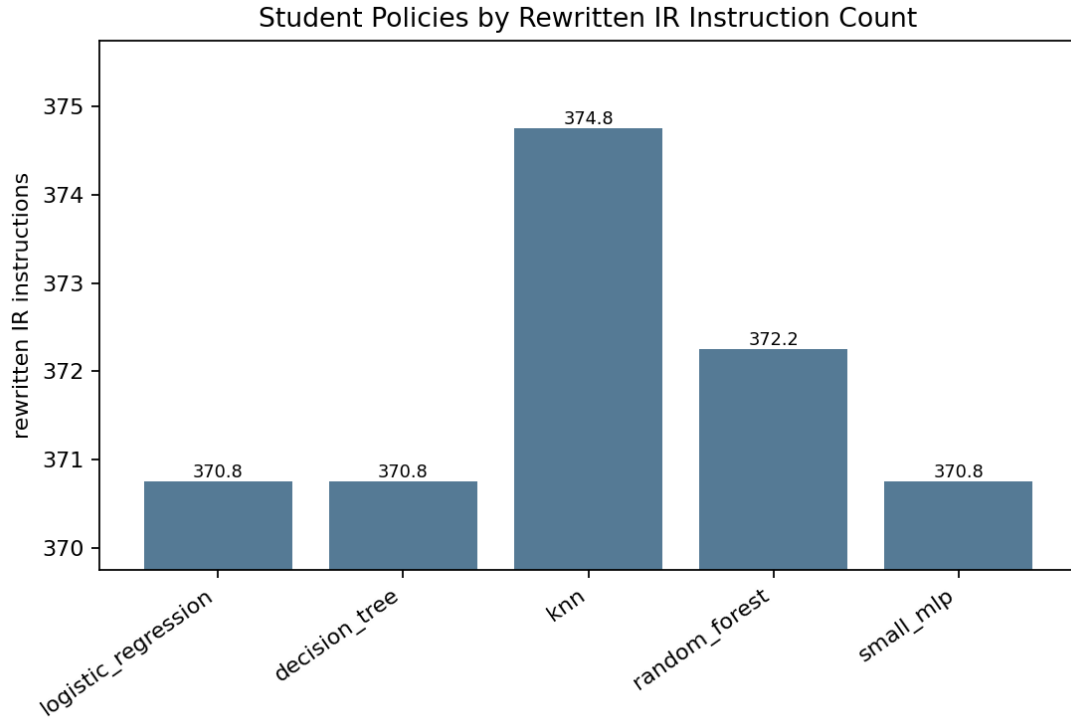


Figure 8.2: Average rewritten LLVM IR instruction count for each student policy.

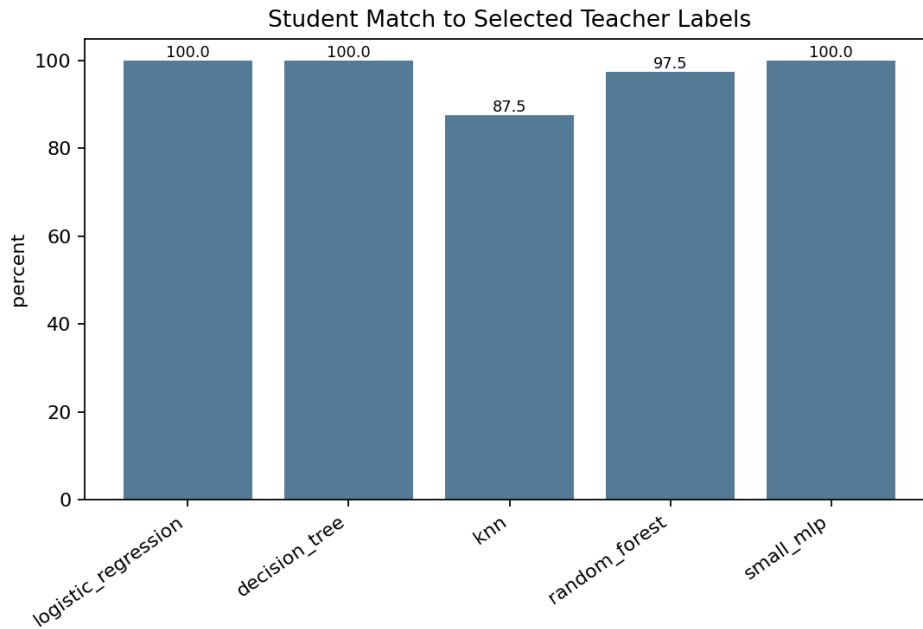


Figure 8.3: Student agreement with the selected teacher labels.

The first comparison follows the BC-Max structure. We evaluate every teacher policy on each module and select the best teacher per module:

Module	Selected teacher	Rewritten IR instructions
cxx11.ll	cost_budget	100
cxx14.ll	cost_budget	27
cxx17.ll	small_callee	123
vector.ll	small_callee	1233

The student models are then evaluated by applying their predicted inline decisions to the IR rewrite step and measuring rewritten IR instruction count:

Model	Teacher-label match	Inline decisions	Avg. rewritten IR instructions	Reduction vs never-inline
logistic_regression	1.000	34	370.75	6.26%
decision_tree	1.000	34	370.75	6.26%
knn	0.875	39	374.75	5.25%
random_forest	0.975	35	372.25	5.88%
small_mlp	1.000	34	370.75	6.26%

The best students by average rewritten IR instruction count are logistic regression, decision tree, and the small MLP. All three reach an average rewritten IR instruction count of 370.75, corresponding to a 6.26% reduction relative to the never-inline baseline when computed from the average instruction counts. When reduction is computed per module and then averaged, the mean reduction is 5.98%, while the median reduction is 6.26%. This distinction explains the small difference between the table value and the per-module summary statistic.

Teacher-label match is only a diagnostic metric. A model can match the selected teacher labels well without necessarily improving the final IR objective. The main evaluation criterion is therefore the rewritten IR instruction count after applying the predicted decisions. In the current run, the models with perfect teacher-label match also obtain the best rewritten IR instruction count, but this need not hold in general.

To add a metric closer to the way compiler papers report results, the pipeline also compiles the standalone sources to native object files and measures the `__text/.text` section size. This Clang measurement complements the LLVM IR instruction-count metric, but it does not measure the learned student policy directly. It is a reference comparison between standard compiler modes, included to show the scale of native code-size changes under ordinary LLVM optimization settings.

Compiler mode	Total text size	Average text size
no_inline_00	59864	3521.41
no_inline_0z	28772	1692.47
llvm_0z	11444	673.18

This mirrors the shape of the offline imitation-learning paper at a smaller scale: compare baselines, select the best baseline per context, train a student from those trajectories, and evaluate

the resulting policy by the objective value. The full paper evaluates real binaries: the MLGO work reports `.text` size reduction against LLVM `-Oz`, while the offline imitation-learning work reports size savings against its base ES/PPO policies on training and test binaries. Reproducing that level of evaluation would require running the learned policy inside the LLVM inliner or otherwise rewriting IR, then compiling to native objects and measuring `.text` or final binary size.

For the DCMTK module `cxx11.11`, the file `out/callgraph_trace.md` shows step by step how the call graph changes after each inline/no-inline decision. The pipeline also writes result plots to `out/figures` and animated per-module call-graph GIFs to `out/gifs`.

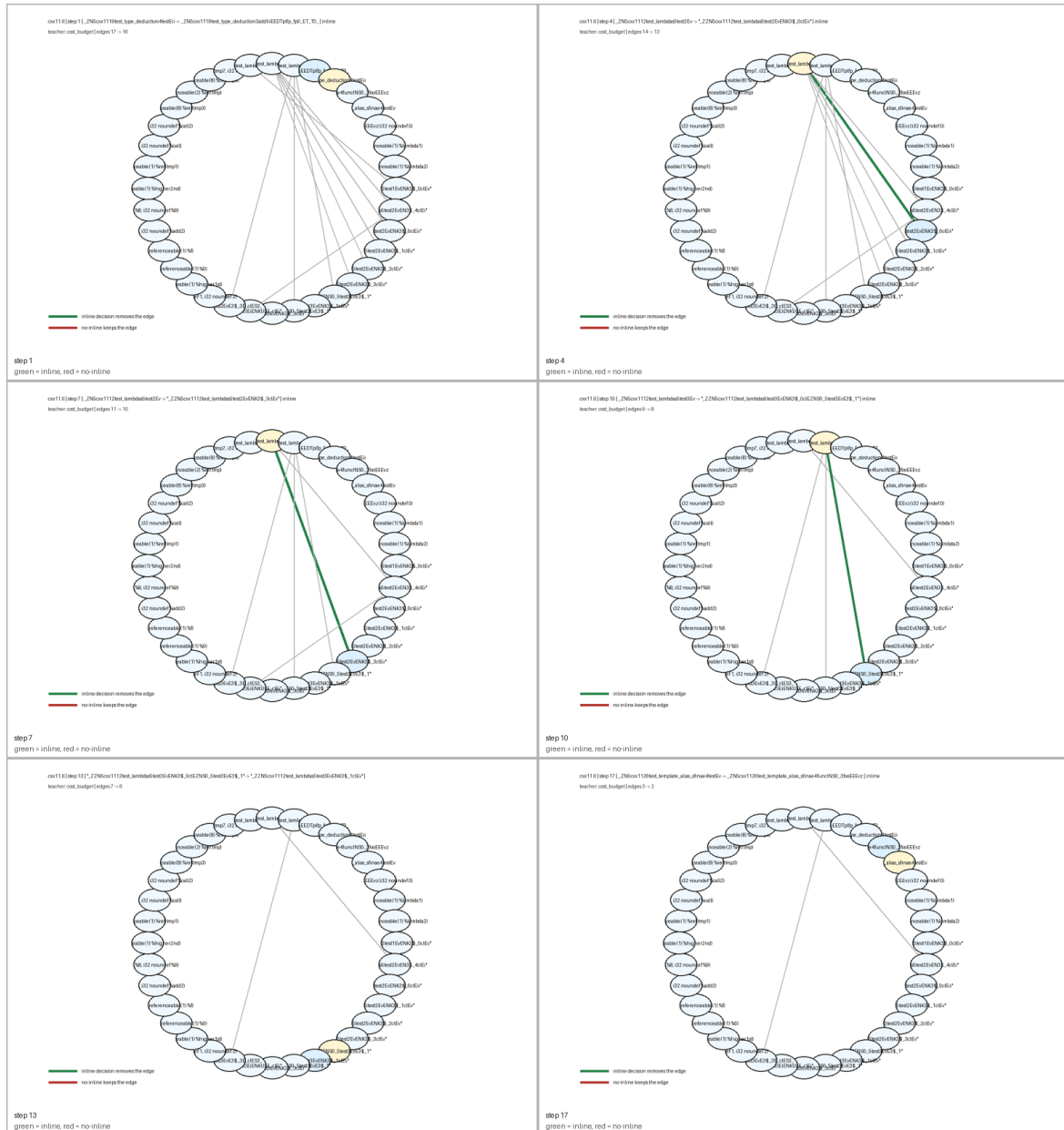


Figure 8.4: Call-graph evolution for `cxx11.11` across six representative decisions. Green edges show inline choices, red would mark no-inline choices, and the graph becomes progressively smaller as edges are removed.

Chapter 9

Limitations

9.1 Small Dataset

The current experiment uses 17 successfully compiled DCMTK source files and 40 local call-sites. This is enough to test the mechanics of the pipeline, but it is too small to support strong claims about generalization. A larger benchmark set would be needed to evaluate whether the learned heuristic transfers across projects, coding styles, and program sizes.

9.2 No Held-Out Generalization Result

The current teacher-label match is computed on the generated behavior-cloning dataset. It is therefore an imitation diagnostic, not a held-out generalization result. A stronger evaluation would split modules into training and test sets, train the student only on the training modules, and apply the learned policy to unseen modules.

9.3 Textual IR Rewriting

This approach rewrites textual LLVM IR by selecting exact callsites, cloning the callee per selected call edge, marking the clone `alwaysinline`, and then running LLVM `opt`. The decision granularity is callsite-level, but the implementation is still a textual rewrite rather than a custom LLVM pass. That means the compiler is guided toward the selected inline edges rather than patched at the inliner internals.

9.4 IR Instruction Count Versus Native Binary Size

The primary metric is rewritten LLVM IR instruction count. This metric is easy to inspect and target-independent, but it is not identical to final native code size. Backend instruction selection, register allocation, alignment, debug information, linking, and target architecture can all affect

native binary size. For this reason, native .text size is reported only as a secondary reference point.

9.5 Teacher Quality

BC-Max can only imitate and combine the behavior available in the teacher set. If all teacher policies are weak, biased, or too similar, the student will also be limited. The current teachers are intentionally simple and inspectable, but they are not equivalent to LLVM’s production inlining heuristic.

9.6 No Full MLGO Reproduction

The prototype is inspired by MLGO and BC-Max, but it is not a reproduction of Google’s infrastructure. A full reproduction would require integration into LLVM’s inliner, large-scale corpus extraction, stronger teacher policies, and evaluation on final native binaries or production-scale benchmarks.

Chapter 10

Conclusions

This thesis studied compiler inlining as an offline policy-learning problem. The implemented prototype compiles C++ files to LLVM IR, extracts callsite-level features, evaluates a set of teacher heuristics, selects the best teacher per module using a BC-Max-style rule, trains student classifiers by behavior cloning, and applies the predicted decisions back to LLVM IR through a rewrite step.

The results show that the full pipeline can be executed end to end on a small DCMTK-based dataset. In the current run, the best student policies reduce rewritten LLVM IR instruction count relative to the never-inline baseline. The native `.text` measurements provide additional context for standard LLVM optimization modes, although they do not yet represent a direct native evaluation of the learned policy.

The main conclusion is methodological: compiler inlining can be framed as a machine-learning workflow in which teacher heuristics generate trajectories and a student model learns a new heuristic from selected behavior. At the same time, the current prototype remains limited by dataset size, lack of held-out generalization, textual IR rewriting, and the absence of full LLVM inliner integration.

Future work should extend the benchmark set, add a train/test split, compare against LLVM's built-in inliner more directly, integrate the learned policy into LLVM as a proper pass or inliner hook, and evaluate native `.text` or final binary size after the complete backend and linking pipeline.



Bibliography

- [1] Teodor V. Marinov, Alekh Agarwal, and Mircea Trofin. “Offline Imitation Learning from Multiple Baselines with Applications to Compiler Optimization.” In: *arXiv preprint arXiv:2403.19462* (2024).
- [2] OFFIS e.V. *DCMTK: DICOM Toolkit*. URL: <https://github.com/DCMTK/dcmTk> (visited on 05/13/2026).
- [3] Mircea Trofin, Yundi Qian, Eugene Brevdo, Zinan Lin, Krzysztof Choromanski, and David Li. “MLGO: A Machine Learning Guided Compiler Optimizations Framework.” In: *arXiv preprint arXiv:2101.04808* (2021).

